Continual Learning: Formulation, Challenges, and Prompt-Based Solutions

1 Introduction to Continual Learning Challenges

1.1 Problem Setting

In the continual-learning (CL) paradigm a single parametric model

$$f_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}, \qquad \boldsymbol{\theta} \in \mathbb{R}^p,$$

is exposed sequentially to a collection of tasks

$$\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_T\}, \qquad \mathcal{D}_t = \{(\mathbf{x}_i^{(t)}, y_i^{(t)})\}_{i=1}^{N_t}.$$

At step t the learner may only access the current dataset $\mathcal{D}_t$ (and possibly a small amount of auxiliary information retained from previous steps). The goal is to optimise a cumulative objective

$$\min_{\boldsymbol{\theta}} \sum_{t=1}^{T} \mathcal{L}\big(f_{\boldsymbol{\theta}}(\mathcal{D}_t)\big), \tag{1}$$

while preserving good performance on all previously seen tasks.

1.1.1 Catastrophic Forgetting

When a model is updated using only the data of task t, the parameters $\boldsymbol{\theta}$ are shifted in a direction that benefits $\mathcal{D}_t$ but may degrade the loss on earlier tasks $\mathcal{D}_1, \ldots, \mathcal{D}_{t-1}$. This phenomenon is termed *catastrophic forgetting*. A common quantitative measure is the *forgetting factor*

$$\mathcal{F}_t \;=\; \max_{k<t} \big[\mathcal{L}\big(f_{\boldsymbol{\theta}_t}(\mathcal{D}_k)\big) - \mathcal{L}\big(f_{\boldsymbol{\theta}_k}(\mathcal{D}_k)\big)\big], \tag{2}$$

where $\boldsymbol{\theta}_k$ denotes the parameters after finishing task k and $\boldsymbol{\theta}_t$ the parameters after the current update. A low $\mathcal{F}_t$ indicates that the model retains knowledge of previous tasks.

1.1.2 Canonical Families of Solutions

Rehearsal-based approaches. These methods allocate a memory buffer $\mathcal{B}$ (often of fixed capacity M) that stores a subset of exemplars from past tasks. The loss in Eq. (1) is replaced by a joint loss over the current mini-batch $\mathcal{B}_t$ and the buffer:

$$\mathcal{L}_t^{\text{rehearsal}} = \frac{1}{|\mathcal{B}_t|} \sum_{(\mathbf{x},y)\in\mathcal{B}_t} \ell\big(f_{\boldsymbol{\theta}}(\mathbf{x}), y\big) + \frac{1}{|\mathcal{B}|} \sum_{(\mathbf{x},y)\in\mathcal{B}} \ell\big(f_{\boldsymbol{\theta}}(\mathbf{x}), y\big), \tag{3}$$

where $\ell(\cdot,\cdot)$ is a task-specific surrogate loss (e.g., cross-entropy).

Architecture-based approaches. These methods allocate disjoint model components (e.g., separate heads, sub-networks, or mask-based pathways) for each task, denoted

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = g_{\boldsymbol{\theta}^{(t)}}(\mathbf{x}), \qquad \boldsymbol{\theta}^{(t)} \cap \boldsymbol{\theta}^{(s)} = \emptyset \ (t \neq s). \tag{4}$$

1.1.3 Prompt-Based Continual Learning (L2P)

Learn to Prompt (L2P) seeks to overcome the two limitations highlighted above: (i) the reliance on stored data and (ii) the necessity of a known task identifier at test time.

Key idea. Instead of storing raw examples, L2P retains a compact set of *learnable prompts*

$$\mathcal{P} = \{\mathbf{p}_1, \ldots, \mathbf{p}_K\}, \qquad \mathbf{p}_k \in \mathbb{R}^d, \tag{5}$$

which are appended (or injected) to the input embedding of each downstream sample. In a transformer-based backbone $h_{\boldsymbol{\theta}}$ the processed representation becomes

$$\tilde{\mathbf{z}} = h_{\boldsymbol{\theta}}\big([\mathbf{p}_1; \ldots; \mathbf{p}_K; \mathbf{x}]\big), \tag{6}$$

where $[\cdot]$ denotes concatenation. The prompts are the only parameters that are updated when a new task arrives; the backbone weights $\boldsymbol{\theta}$ are frozen after an initial pre-training phase. Consequently, the memory footprint of $\mathcal{P}$ is orders of magnitude smaller than a full rehearsal buffer.

Training objective. For task t, the prompt set is adapted by minimising a standard supervised loss on the current data $\mathcal{D}_t$, while a *prompt-selection regulariser* ensures that the prompts remain useful for past tasks:

$$\mathcal{L}_t^{\text{L2P}} = \frac{1}{|\mathcal{D}_t|} \sum_{(\mathbf{x},y)\in\mathcal{D}_t} \ell\big(g_{\mathcal{P}}(\mathbf{x}), y\big) + \lambda \underbrace{\frac{1}{K}\sum_{k=1}^{K} \big\|\mathbf{p}_k^{(t)} - \mathbf{p}_k^{(t-1)}\big\|_2^2}_{\text{stability term}}, \tag{7}$$

where $g_{\mathcal{P}}(\cdot)$ denotes the downstream classifier that receives the prompt-augmented representation, and λ trades off plasticity versus stability.

Task-agnostic inference. At test time the model does not require a task label. Instead, a lightweight *prompt selector* ϕ computes a relevance score for each stored prompt based on the input:

$$s_k = \phi(\mathbf{x})^\top \mathbf{p}_k, \qquad \tilde{\mathbf{p}} = \sum_{k=1}^{K} \alpha_k \mathbf{p}_k, \ \alpha_k = \frac{\exp(s_k)}{\sum_j \exp(s_j)}. \tag{8}$$

The aggregated prompt $\tilde{\mathbf{p}}$ is then injected together with $\mathbf{x}$, resulting in a single forward pass that automatically adapts to the underlying task.

1.1.4 Summary of Advantages

- **Privacy-preserving:** No raw examples are retained; only a small set of learnable vectors $\mathcal{P}$ is stored.
- **Scalable memory:** $|\mathcal{P}| = K \ll M$, where M is the typical rehearsal-buffer size.
- **Task-free deployment:** The selector ϕ produces a data-dependent prompt, removing the need for an explicit task identifier at inference.
- **Constant model size:** The backbone $h_{\boldsymbol{\theta}}$ is frozen after pre-training, so the total number of trainable parameters grows only with K.

Questions

1. **Definition of Catastrophic Forgetting (Easy).**
   Define catastrophic forgetting in continual learning. Explain why training a model sequentially on tasks $\{\mathcal{D}_1, \ldots, \mathcal{D}_T\}$ typically leads to degradation on earlier tasks.

2. **Rehearsal vs. Architecture Strategies (Easy–Medium).**
   Compare rehearsal-based methods with architecture-based methods. For each family list at least two advantages and two limitations, focusing on memory usage, scalability, and the requirement of task identifiers during inference.

3. **Why Objective (1) Is Insufficient (Medium).**
   Discuss why minimizing the cumulative loss $\sum_{t=1}^{T} \mathcal{L}\big(f_{\boldsymbol{\theta}}(\mathcal{D}_t)\big)$ alone does not guarantee continual-learning performance. Use a simple two-task linear-classifier example to illustrate the role of the forgetting factor $\mathcal{F}_t$.

4. **Numerical Forgetting Factor (Easy).**
   Given loss values: after task 1, $\mathcal{L}_1 = 0.2$; after task 2, loss on $\mathcal{D}_1$ becomes 0.6; after task 3, loss on $\mathcal{D}_1$ becomes 0.4. Compute $\mathcal{F}_2$ and $\mathcal{F}_3$ and comment on what they reveal about retention.

5. **Complexity of Rehearsal Loss (Medium).**

   (a) Derive the computational complexity of evaluating the joint rehearsal loss (3) for one training step in terms of buffer size M and mini-batch size $|\mathcal{B}_t|$.

   (b) Identify three practical scenarios where this complexity is a bottleneck (e.g., strict privacy, edge-device storage, many tasks) and propose a high-level mitigation for each.

6. **Zero Forgetting with Disjoint Parameters (Hard).**

   (a) Show mathematically that if $\boldsymbol{\theta}^{(t)} \cap \boldsymbol{\theta}^{(s)} = \emptyset$ for $t \neq s$, then the forgetting factor $\mathcal{F}_t = 0$ for all t.

   (b) Discuss the trade-off between this guarantee and the $\mathcal{O}(T)$ growth of the model size. Argue whether architecture-based methods are viable on devices with a hard memory cap.

7. **Memory Footprint of Prompts vs. Rehearsal Buffer (Easy–Medium).**
   A backbone has 10^7 parameters. Each prompt vector has dimension $d = 768$ and $K = 20$. A rehearsal buffer stores $M = 2000$ image embeddings of the same dimension. Compute the memory (in number of float parameters) required for the prompts and for the buffer, and give the ratio $\frac{\text{prompt memory}}{\text{buffer memory}}$. Discuss how prompts alleviate privacy concerns compared with raw data storage.

8. **Stability–Plasticity Gradient (Medium).**

   (a) Derive the gradient of the stability term $\frac{\lambda}{K} \sum_{k=1}^{K} \|\mathbf{p}_k^{(t)} - \mathbf{p}_k^{(t-1)}\|_2^2$ with respect to a single prompt $\mathbf{p}_k^{(t)}$.

   (b) Explain how varying λ balances plasticity and stability. Propose an experimental protocol (e.g., grid search with five-task benchmark) to select an appropriate λ.

9. **Selector as Matrix–Vector Product (Hard).**
   Assume $\phi(\mathbf{x}) = \mathbf{W}\mathbf{x}$ with $\mathbf{W} \in \mathbb{R}^{d\times d}$. Show that the aggregated prompt $\tilde{\mathbf{p}}$ from Eq. (8) can be written as $\tilde{\mathbf{p}} = \mathbf{A}\mathbf{x}$ for some matrix $\mathbf{A}$ that depends on $\{\mathbf{p}_k\}$ and $\mathbf{W}$. Analyse the extra computational cost of computing $\tilde{\mathbf{p}}$ compared with a plain forward pass, and state for which values of K and d the overhead becomes negligible.

10. **Theoretical Bound on Forgetting (Hard).**
    Sketch a proof that, assuming the backbone $h_{\boldsymbol{\theta}}$ is L-Lipschitz, the forgetting factor after task t satisfies
    $$\mathcal{F}_t \le \frac{\lambda}{K}\sum_{k=1}^{K} \|\mathbf{p}_k^{(t)} - \mathbf{p}_k^{(t-1)}\|_2^2 \cdot L.$$
    List any additional assumptions (e.g., bounded loss, smoothness) you need, and discuss the practical implications for choosing the number of prompts K and the regularisation weight λ.

11. **Ablation Study Design (Hard).**
    Propose an ablation study that isolates the contributions of (i) prompt set size K, (ii) stability regulariser λ, and (iii) selector ϕ.
    (a) Define three experimental settings, each removing or altering one component.
    (b) Predict how each setting will affect (a) the forgetting factor $\mathcal{F}_t$ and (b) overall test accuracy on a benchmark such as Split-CIFAR-100. Justify your predictions using the theory above.

12. **Multimodal Extension of L2P (Hard).**
    Imagine a continual-learning system that must handle tasks from different modalities (images, text, audio).
    (a) Propose a modification to the prompt architecture that allows modality-specific prompts while preserving privacy and task-free inference.
    (b) Critically discuss challenges such as prompt interference across modalities and the design of a universal selector ϕ that can operate on heterogeneous inputs.

13. **Hybrid Prompt-Architecture Approach (Hard).**
    Design a hybrid method for a setting where replay buffers are disallowed for privacy reasons but knowledge retention is required. Combine (i) prompt-based conditioning (as in L2P) with (ii) lightweight architectural adaptation (e.g., sparsely activated expert modules).
    (a) Outline the training pipeline, including how prompts and expert modules are updated.
    (b) Describe the inference procedure, emphasizing task-free operation.
    (c) Discuss trade-offs in computational overhead, memory usage, and forgetting mitigation compared with pure prompt-based or pure architectural methods.

2 Prompt Pool Mechanism

Motivation

In continual learning a model must assimilate a stream of tasks $\{\mathcal{T}_1, \mathcal{T}_2, \dots\}$ while preserving knowledge acquired from earlier tasks. Classical approaches either (i) *share a single prompt* $\mathbf{P}$ across all tasks, or (ii) allocate a *task-specific prompt* $\mathbf{P}^{(t)}$ that is activated only when the task identifier t is known. A shared prompt suffers from **interference**: gradients arising from distinct tasks are projected onto the same parameter subspace, aggravating catastrophic forgetting. Conversely, task-specific prompts require an external task label, which contradicts the desideratum of *task-agnostic* continual learning.

The **Prompt Pool** paradigm balances these extremes by maintaining a collection of reusable prompt embeddings that can be dynamically combined for each input instance. This yields fine-grained, instance-wise knowledge sharing without explicit task signals.

Definition of a Prompt Pool

Let $\mathcal{P}$ denote a *prompt pool* consisting of M learnable prompt vectors:

$$\boxed{\mathcal{P} = \{\, \mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_M \,\}} \tag{1}$$

Each $\mathbf{P}_i \in \mathbb{R}^{d_p}$ is a *prompt embedding* that can be prepended (or appended) to the token sequence of any downstream model. The pool is jointly optimized together with the base model parameters during continual training.

Instance-wise Prompt Composition

For a given input example x, its raw embedding (e.g., the output of a frozen tokenizer or an image encoder) is denoted $\mathbf{x}_e \in \mathbb{R}^{d_e}$. Instead of attaching a single prompt, we select a subset of N distinct prompts from $\mathcal{P}$:

$$\mathcal{S}(x) = \{s_1, s_2, \dots, s_N\} \subseteq \{1, \dots, M\}, \qquad 1 < N < M.$$

The indices s_k may be obtained by a lightweight selector network, a similarity search, or a learned gating mechanism. The adapted embedding fed to the backbone model is

$$\boxed{\mathbf{x}_p = \left[\, \mathbf{P}_{s_1} ; \mathbf{P}_{s_2} ; \dots ; \mathbf{P}_{s_N} ; \mathbf{x}_e \,\right]} \tag{2}$$

where $[\cdot]$ denotes concatenation along the token dimension. The resulting sequence contains N prompt tokens followed by the original data token(s).

Properties and Benefits

1. **Reduced Interference.** Because each instance draws only a sub-pool, gradients from disparate tasks are distributed across different $\mathbf{P}_i$, lowering the probability that updates for one task overwrite the representation needed for another.
2. **Task-agnostic Operation.** No explicit task identifier is required; the selector implicitly discovers which prompts are useful for the current input, enabling true continual learning in open-world settings.
3. **Fine-grained Knowledge Sharing.** Prompts can be reused across tasks whenever they capture useful sub-skills (e.g., "detect edges", "recognize colour patterns"). The composition $\mathcal{S}(x)$ implements a form of *compositional memory* where high-level behaviour emerges from low-level prompt combinations.

4. **Scalability.** The pool size M can be large while the active set N remains modest ($N \ll M$), keeping the computational overhead proportional to N rather than M.

Relation to Catastrophic Forgetting

In the classical continual-learning formulation, catastrophic forgetting arises when the parameter update $\Delta\theta$ for a new task $\mathcal{T}_t$ significantly deviates from the optimal region for previous tasks $\{\mathcal{T}_1, \ldots, \mathcal{T}_{t-1}\}$. By confining most task-specific information to a small, dynamically chosen subset of prompts, the bulk of the backbone parameters θ_{base} experience milder, more stable gradients. Consequently the risk of $\Delta\theta_{\text{base}}$ causing forgetting is mitigated, while the prompt pool itself can be regularised (e.g., using orthogonal constraints or replay buffers) to preserve useful prompts over time.

Learning the Selector

A practical implementation often employs a differentiable selector g_ϕ parameterised by ϕ:

$$\mathbf{a} = g_\phi(\mathbf{x}_e) \in [0,1]^M, \qquad s_k = \operatorname*{arg\,top}_N(\mathbf{a}),$$

where $\arg\operatorname{top}_N$ returns the indices of the N largest activation values. The overall loss for an input x with label y may be written as

$$\mathcal{L}(x, y) = \ell\big(f(\mathbf{x}_p; \theta_{\text{base}}),\, y\big) + \lambda\, \Omega(\mathbf{P}_1, \ldots, \mathbf{P}_M),$$

with ℓ a task-specific cross-entropy or regression loss and Ω a regulariser (e.g., prompt orthogonality or replay-based penalty). Gradients propagate through both the selected prompts and the selector, allowing the pool to self-organise into a repertoire of reusable skills.

Summary

The Prompt Pool framework introduces a middle ground between a monolithic shared prompt and a proliferation of task-specific prompts. By defining a finite set $\mathcal{P}$ of reusable prompt embeddings and constructing instance-wise adapted embeddings $\mathbf{x}_p$ through selective concatenation, the method supports:

- *instance-level composition* of knowledge,
- *task-agnostic continual learning*,
- *reduction of catastrophic forgetting* via distributed parameter updates.

Questions

1. **Motivation – Comparative analysis (medium).**
   Contrast the three prompting strategies—single shared prompt, task-specific prompts, and a prompt pool—on: (i) susceptibility to catastrophic forgetting, (ii) need for task identifiers, and (iii) scalability to many tasks. Which strategy offers the most favourable trade-off for task-agnostic continual learning and why?

2. **Dimensionality calculation (easy).**
   For a prompt pool with $M = 100$ prompts, each $\mathbf{P}_i \in \mathbb{R}^{d_p}$, $d_p = 20$, and a raw embedding $\mathbf{x}_e \in \mathbb{R}^{d_e}$ with $d_e = 256$, compute the dimensionality of the adapted embedding $\mathbf{x}_p$ when $N = 5$ prompts are concatenated. Discuss the impact on computational load compared with concatenating the whole pool ($N = M$).

3. **Lightweight selector design (medium).**
   Propose a selector g_ϕ suitable for a Vision-Transformer backbone. Specify (i) the layer types (e.g., linear, depth-wise conv), (ii) the activation function, and (iii) how the architecture enables fast top-N selection at inference time.

4. **Interference mitigation – Critical evaluation (hard).**
   Explain why selecting a subset $\mathcal{S}(x)$ of prompts reduces gradient interference between tasks. Under what conditions could this mitigation fail (e.g., highly overlapping tasks)? Suggest a regularisation technique to alleviate the failure.

5. **Task-agnostic deployment on a mobile robot (hard).**
   Sketch the end-to-end pipeline for a robot that receives raw visual sensor data, uses a prompt-pool system without any task label, and produces a prediction (e.g., navigation decision). Emphasise the role of the selector and how the system adapts online.

6. **Fine-grained knowledge sharing – Synthesis (medium).**
   Identify three hypothetical sub-skills that could be encoded in individual prompts (e.g., edge detection, colour constancy, texture synthesis). Describe how a particular combination of these prompts could yield a higher-level capability such as recognising reflective metallic objects.

7. **Scalability under memory budget (easy).**
   With a memory budget of $K = 10$ MiB for prompt embeddings, each stored as 32-bit floats, $d_p = 64$ and $N = 4$, compute the maximum number of prompts M_{max} that can be stored. Discuss the trade-off between increasing M versus increasing N given the same budget.

8. **Effect on backbone updates – Derivation (hard).**
   Starting from the standard continual-learning update $\Delta\theta = -\eta\nabla_\theta\mathcal{L}$, show how isolating most task-specific information in the prompt pool changes the effective update on the backbone parameters θ_{base}. What implication does this have for the Fisher information matrix used in regularisation-based CL methods (e.g., EWC)?

9. **Parallel top-N on GPU (medium).**
   Given a batch of $B = 32$ inputs, each yielding an activation vector $\mathbf{a}^{(i)} \in [0,1]^M$, outline an algorithm that computes the top-N indices for all batch elements in parallel on a GPU. Explain why such parallelisation is crucial for training efficiency.

10. **Regularisation strategies comparison (medium).**
    Compare (a) an orthogonal constraint $\Omega_{\text{ortho}} = \sum_{i\neq j} \|\mathbf{P}_i^\top \mathbf{P}_j\|^2$ with (b) a replay-based penalty that constrains prompt activations to stay close to stored exemplars. Which is more appropriate when the prompt pool must constantly adapt to a drifting task distribution? Justify your choice.

11. **Hierarchical prompt pools – Open-ended design (hard).**
    Propose an extension that organises prompts hierarchically (e.g., groups or a tree). Discuss how this hierarchy would affect selector design, memory consumption, and the system's ability to transfer knowledge across distant tasks.

12. **Summative reflection (easy).**
    In no more than three sentences, summarise how the Prompt Pool framework simultaneously tackles (i) interference, (ii) the need for task identifiers, and (iii) scalability, linking each point to the relevant equation or property from the theory.

3 Instance-wise Prompt Querying

Key-Value Retrieval in L2P

In the Learning to Prompt (L2P) framework the prompt pool is organised as a *key-value* store. Each prompt P_i (the *value*) is coupled with a learnable embedding

$$k_i \in \mathbb{R}^{D_k}, \qquad i = 1, \ldots, M,$$

where M denotes the total number of prompts in the pool and D_k is the dimensionality of the key space. The key vectors are the parameters that are trained (e.g. by back-propagation) while the prompts themselves remain fixed once instantiated. This separation enables the system to retrieve the most relevant prompts for a new input without altering the prompts themselves, thereby preserving previously acquired knowledge.

Query Construction

Given an incoming sample x, L2P first extracts a compact representation of the input by means of a frozen pre-trained model (typically a vision or language backbone). Formally,

$$q(x) = f(x)[0, :] \in \mathbb{R}^{D_k},$$

where f denotes the backbone network and the notation $[0, :]$ extracts the feature vector associated with the CLS token (or its analogue). Because f is frozen, the query function $q(\cdot)$ is **task-agnostic**: it depends only on the raw input distribution, not on any task-specific label or loss.

Matching Score and Distance Metric

To assess the compatibility between a query $q(x)$ and a key k_i a similarity (or distance) function $\gamma(\cdot, \cdot)$ is defined. In L2P the most common choice is the cosine distance

$$\gamma_{\cos}(a, b) = 1 - \frac{a^\top b}{\|a\|_2 \, \|b\|_2},$$

which is bounded in $[0, 2]$ and treats vectors that point in the same direction as highly similar (score close to 0). Other admissible choices include Euclidean distance or learned metrics, but the cosine variant enjoys scale invariance and computational simplicity.

Top-N Retrieval Procedure

The goal of retrieval is to select a subset of N prompts whose keys best match the query. Let $\{s_i\}_{i=1}^N$ be the indices of the chosen prompts, with $s_i \in \{1, \ldots, M\}$. The selection problem can be written as the optimisation

$$\{s_i\}_{i=1}^N = \underset{\substack{\{t_i\}_{i=1}^N \subseteq [1,M] \\ t_i \neq t_j \ \forall i \neq j}}{\arg\min} \sum_{i=1}^{N} \gamma\big(q(x),\, k_{t_i}\big). \tag{3}$$

Equation (3) simply asks for the N keys with the smallest matching scores (i.e. largest cosine similarity). In practice this is implemented by computing $\gamma(q(x), k_i)$ for all i and performing an efficient partial sort (often via `torch.topk` or `numpy.argpartition`) to obtain the indices of the top-N matches:

$$K_x = \{k_{s_1}, \ldots, k_{s_N}\}, \qquad P_x = \{P_{s_1}, \ldots, P_{s_N}\}.$$

Task-Agnostic Memory Access

Because the retrieval depends only on the similarity between the frozen feature of the raw input and the learned keys, no task identifier or label is required. This property yields two important benefits for continual learning:

1. **Task-invariant storage:** Prompt vectors can be reused across tasks as long as their keys capture the underlying data modality. Consequently, when a new task arrives the system does not need to allocate a dedicated prompt bank; it can simply query the existing pool and obtain the most compatible prompts.
2. **Mitigation of catastrophic forgetting:** Since the prompts themselves are never overwritten during retrieval, earlier knowledge embedded in the pool remains intact. The only parameters that adapt are the keys k_i, which evolve to better align with the distribution of incoming samples while preserving the relative positions of previously learned keys.

Relation to the Prompt Pool Mechanism

Recall from the previous section that the prompt pool acts as an external memory where each prompt stores a "piece of knowledge" (e.g. a class prototype or a task-specific feature transformation). The key-value mechanism described here furnishes a *retrieval interface* to that memory:

- **Storing:** During training on a task, the model updates both the content of the prompts (through gradient descent on downstream loss) and the corresponding keys (so that future queries will retrieve them).
- **Retrieving:** At inference time (or when processing a new task), the query function extracts a representation of the current input, matches it to the stored keys, and selects the most relevant prompts to be prepended to the backbone.

Thus the key-value retrieval is the operational bridge that transforms the static prompt pool into a dynamic, task-agnostic memory system capable of supporting continual learning without the need for explicit task labels.

Summary of the Retrieval Pipeline

1. **Feature extraction:** $q(x) = f(x)[0, :]$ using a frozen backbone.
2. **Similarity computation:** Evaluate $\gamma(q(x), k_i)$ for all keys.
3. **Top-N selection:** Solve Equation (3) to obtain index set $\{s_i\}_{i=1}^{N}$.
4. **Prompt aggregation:** Retrieve prompts $\{P_{s_i}\}_{i=1}^{N}$ and concatenate (or otherwise combine) them with the input representation before forwarding to the downstream model.

Questions

1. **Conceptual analysis (Medium).**
   Explain why the keys k_i are the only learnable parameters in the L2P retrieval module, while the prompts P_i remain fixed after instantiation. Discuss the consequences of this design choice for
   (a) continual-learning stability, and
   (b) the ability to reuse prompts across different tasks.

2. **Derivation of the matching score (Easy).**
   Show that minimising the cosine distance

$$\gamma_{\cos}(q(x), k_i) = 1 - \frac{q(x)^\top k_i}{\|q(x)\|_2 \|k_i\|_2}$$

   is equivalent to maximising the cosine similarity between the query and the key. Provide a short proof and explain why the distance formulation is preferred in the optimisation problem of Equation (3).

3. **Application of the top-N retrieval (Easy).**
   Suppose a prompt pool contains $M = 10\,000$ keys of dimension $D_k = 256$. For a new input x you compute the cosine distances to all keys and obtain the 5 smallest values: $0.03, 0.05, 0.07, 0.09, 0.11$.

   (a) What are the corresponding cosine similarities?

   (b) If the retrieval system were to use Euclidean distance instead, would the same set of indices necessarily be selected? Justify your answer qualitatively.

4. **Critical evaluation of task-agnostic memory access (Hard).**
   Identify at least two scenarios in which the task-agnostic property of the retrieval mechanism could become a limitation for continual learning. For each scenario propose a modification to the L2P pipeline that could mitigate the limitation.

5. **Design problem (Medium).**
   Design an experiment to compare the effect of three different similarity metrics—(i) cosine distance, (ii) Euclidean distance, and (iii) a learned Mahalanobis distance—on the quality of retrieved prompts. Outline:

   - the dataset and tasks you would use;
   - the evaluation metric(s) (e.g., downstream accuracy, forgetting rate);
   - how you would keep the comparison fair (e.g., same number of keys, same training budget).

6. **Algorithmic implementation (Easy).**
   Write pseudocode (Python-like) that implements the retrieval step described in the summary pipeline, explicitly showing:

   (a) extraction of the query vector $q(x)$;

   (b) computation of the cosine distances to all keys;

   (c) use of an efficient partial-sort routine to obtain the indices of the top-N matches;

   (d) retrieval and concatenation of the corresponding prompts.

   Comment on the computational complexity with respect to M (size of the pool) and N (number of retrieved prompts).

7. **Quantitative reasoning (Medium).**
   Assume the cosine distances for all keys follow a uniform distribution on $[0, 2]$. Approximate the expected value of the minimum distance among the top-N retrieved keys, i.e. $\mathbb{E}[\min\{\gamma_i\}_{i=1}^{N}]$. Use this approximation to discuss how increasing N influences the relevance of the retrieved prompts.

8. **Interpreting the role of the frozen backbone (Hard).**
   Discuss how the choice of the pretrained backbone f (e.g., a vision transformer pretrained on ImageNet vs. a language model pretrained on a large text corpus) influences the geometry of the query vectors $q(x)$. What implications does this have for the learnability of the keys k_i and for the overall retrieval performance?

9. **Synthesis and future direction (Hard).**
   Propose an extension of the key-value retrieval mechanism that incorporates temporal information (e.g., timestamps of when prompts were learned) to improve continual-learning performance. Describe how you would modify the similarity computation and the selection objective, and hypothesise the potential benefits and drawbacks of your approach.

4 Optimization and Training Objective

4.1 Dual-Term Objective for Prompt-Pooled Continual Learning

In the prompt-pool framework the learner optimises three groups of parameters simultaneously:

- $P = \{p_1, \ldots, p_{|P|}\}$ – the *prompt pool*, i.e. the set of learnable prompt vectors that are stored as values in a key-value memory.
- $K = \{k_1, \ldots, k_{|K|}\}$ – the *key bank* associated with the prompts. Each key k_{s_i} lives in the same latent space as a query vector $q(x)$ and is used to retrieve the most relevant prompt(s) for an input x.
- ϕ – the parameters of the classifier head $g_\phi(\cdot)$ that maps the (prompt-augmented) token representation to class logits.

The backbone visual encoder, a Vision Transformer (ViT) denoted $f_r^{\text{avg}}(\cdot)$, is kept frozen throughout training. Freezing the backbone preserves the generic visual features that have been pretrained on a large corpus, while learning P, K and ϕ enables the system to adapt to a stream of tasks without incurring catastrophic forgetting.

4.2 Formal Statement of the Loss

The overall objective is a *dual-term loss*:

$$\min_{P,K,\phi} \underbrace{\mathcal{L}\big(g_\phi\big(f_r^{\text{avg}}(x_p)\big),\, y\big)}_{\text{(i) classification loss}} + \lambda \underbrace{\sum_{k_{s_i} \in K_x} \gamma\big(q(x),\, k_{s_i}\big)}_{\text{(ii) query-key alignment}} \tag{1}$$

where

x_p the input image x after it has been *prompt-augmented*;

y the ground-truth class label for x;

$\mathcal{L}(\cdot,\cdot)$ the standard softmax cross-entropy loss;

$q(x)$ the query vector obtained from the frozen representation $f_r^{\text{avg}}(x)$;

K_x the subset of keys retrieved for x (e.g. top-M nearest neighbours);

$\gamma(\cdot,\cdot)$ a similarity or distance function such as the negative cosine distance

$$\gamma\big(q, k\big) = -\frac{q^\top k}{\|q\|\,\|k\|},$$

or a squared Euclidean distance;

λ a scalar hyper-parameter that controls the trade-off between the two terms.

4.3 Interpretation of the Two Terms

(i) Classification loss. Term (i) forces the classifier head ϕ to produce correct predictions from the feature map that already incorporates the selected prompts. Because the backbone is frozen, all task-specific adaptation must be realised through P and ϕ.

(ii) Surrogate query-key loss. Term (ii) encourages the key vectors to be close to the queries they are intended to serve. Minimising $\gamma\big(q(x), k_{s_i}\big)$ makes k_{s_i} more likely to be retrieved for inputs whose queries are similar, thereby pulling the associated prompt p_{s_i} into the forward pass for those inputs. This tight coupling between the external memory and the instance-wise retrieval process mitigates catastrophic forgetting.

4.4 Optimization Procedure

All learnable parameters $\{P, K, \phi\}$ are updated with the Adam optimiser [**?**]. For each mini-batch $\{(x^{(b)}, y^{(b)})\}_{b=1}^{B}$:

1. Compute the frozen representation $z^{(b)} = f_r^{\text{avg}}(x^{(b)})$.
2. Obtain the query $q^{(b)} = \mathcal{Q}(z^{(b)})$, where $\mathcal{Q}$ is the linear projection that defines $q(\cdot)$.
3. Retrieve the top-M keys $K_{x^{(b)}}$ nearest to $q^{(b)}$ under the similarity measure used in γ.
4. Assemble the prompt-augmented token sequence $x_p^{(b)}$ by concatenating the prompts $\{p_{s_i}\}$ whose keys belong to $K_{x^{(b)}}$.
5. Forward $x_p^{(b)}$ through the frozen ViT, obtain logits $\hat{y}^{(b)} = g_\phi\big(f_r^{\text{avg}}(x_p^{(b)})\big)$, and compute the cross-entropy loss.
6. Compute the surrogate loss $\sum_{k_{s_i} \in K_{x^{(b)}}} \gamma\big(q^{(b)}, k_{s_i}\big)$.
7. Form the total loss according to Eq. (1), back-propagate gradients *only* through P, K, and ϕ, and update them with Adam.

Gradients are blocked from flowing into the frozen ViT, so optimisation focuses exclusively on shaping the external memory (keys and prompts) and the classifier.

4.5 Role of the Trade-off Parameter λ

- **Large** λ – forces tight query-key alignment, yielding precise prompt retrieval but risking over-fitting to the memory structure and hurting classification accuracy.
- **Small** λ – reduces the influence of the surrogate term, letting the classifier rely more on frozen ViT features, which may degrade the ability to exploit task-specific prompts for out-of-distribution tasks.

In practice λ is tuned on a held-out validation stream that mimics the continual-learning scenario.

4.6 Connection to Instance-Wise Prompt Querying

The surrogate loss directly penalises mismatches between queries and their retrieved keys, encouraging a geometry where semantically similar inputs map to nearby keys and thus retrieve the same or similar prompts. This self-organising behaviour preserves earlier knowledge (prompts remain reachable) while allowing new prompts to be added without altering the frozen backbone.

4.7 Summary

The dual-term loss in Eq. (1) provides a principled training objective for a continual-learning system that leverages a frozen Vision Transformer and an external prompt pool. The first term guarantees discriminative performance on the current task, whereas the second term enforces a tight coupling between query vectors and memory keys, ensuring that the instance-wise retrieval mechanism remains effective throughout the lifelong learning trajectory. Optimisation with Adam updates only the mutable components (P, K, ϕ), preserving the stability of the pre-trained visual encoder while granting the flexibility needed to acquire new knowledge incrementally.

Questions

1. **(Analysis)** Derive the gradient of the total loss in Eq. (1) with respect to a single prompt vector $p_j \in P$. Explicitly show how the gradient separates into contributions from the classification term and the query-key alignment term. Discuss why gradients do *not* flow into the frozen ViT $f_r^{\text{avg}}(\cdot)$.

2. **(Analysis)** Consider a scenario where the similarity function γ is the negative cosine distance. Explain how the choice of γ influences the geometry of the key space during training. What would be the effect of replacing γ with a squared Euclidean distance on the retrieved prompts?

3. **(Evaluation)** The hyper-parameter λ governs the trade-off between classification accuracy and query-key alignment. Propose a systematic validation procedure to select an appropriate λ for a continual-learning stream of T tasks. Justify how your procedure balances the two competing desiderata and mitigates over-fitting to either term.

4. **(Evaluation)** Suppose after several tasks the model exhibits a drop in accuracy on early tasks while the surrogate loss remains low. Diagnose potential reasons for this phenomenon in terms of the prompt-pool dynamics and suggest two concrete remedial strategies.

5. **(Application)** You are given a new downstream task whose data distribution deviates significantly from the pre-training corpus. Outline how you would adapt the prompt-pool framework (i.e. modify P, K, ϕ, λ, and M) to maximise performance while preserving knowledge of previous tasks.

6. **(Application)** Design an experiment to compare three variants of the key retrieval mechanism: (a) top-M nearest neighbours under cosine similarity, (b) softmax-weighted attention over all keys, and (c) a learned hashing function. Specify the metrics you would record, the hypotheses you aim to test, and how the dual-term loss would need to be altered, if at all, for each variant.

7. **(Application)** In a resource-constrained setting (limited GPU memory), you can store at most $|P| = 32$ prompts. Describe a principled method to decide which prompts to retain or discard when a new task arrives, making use of the learned key vectors and the surrogate loss values.

8. **(Conceptual)** Explain how the surrogate query-key loss acts as a form of "self-organising memory" and how this mechanism mitigates catastrophic forgetting in continual learning.

9. **(Conceptual)** Discuss the advantages and possible drawbacks of keeping the Vision Transformer backbone frozen while updating P, K, and ϕ with Adam. In what situations might fine-tuning the backbone be preferable?

10. **(Conceptual)** The prompt-augmented token sequence x_p is formed by concatenating selected prompts to the original token stream. Analyse how the number of concatenated prompts (i.e., the value of M) can affect both the representational capacity of the model and the computational cost during inference.

11. **(Synthesis)** Imagine extending the current framework to a multimodal setting where inputs may be images, text, or both. Propose modifications to the dual-term loss, the definition of queries $q(\cdot)$, and the key-value memory to accommodate heterogeneous modalities while preserving the continual-learning benefits.

12. **(Synthesis)** Critique the use of a single scalar λ for balancing the two loss terms throughout an entire lifelong learning trajectory. Suggest an adaptive scheme (e.g., curriculum-based, task-aware, or loss-scheduling) that dynamically adjusts λ, and argue how it could improve overall performance.

5 Experimental Results and Benchmarks

Empirical Evaluation of L2P

In this section we present a systematic empirical study of the **Learning to Prompt** (L2P) framework. The experiments are designed to answer two central questions:

1. How does L2P compare with the current state-of-the-art (SOTA) continual-learning algorithms under a variety of benchmark regimes?
2. To what extent can L2P retain high performance *without* relying on an explicit rehearsal buffer, i.e. purely through the Prompt Pool mechanism and the loss-based optimisation described in the preceding sections?

5.1 Benchmarks and Protocols

Datasets. Four widely used continual-learning benchmarks are employed:

Split CIFAR-100 A class-incremental split of CIFAR-100 into $T = 10$ tasks, each comprising 10 disjoint classes. The standard protocol presents one task at a time and evaluates after each task.

5-Dataset Sequence The sequence {CIFAR-10, MNIST, Fashion-MNIST, SVHN, notMNIST} (each treated as a separate task). This setting mixes image modalities and digit versus object domains, stressing cross-domain transfer.

CORe50 (Domain-incremental) 50 object categories recorded in 8 different acquisition conditions (lighting, background, viewpoint). Each condition constitutes a new domain, testing robustness to distribution shift.

Gaussian-scheduled CIFAR-100 A *task-agnostic* scenario where the class distribution evolves according to a Gaussian schedule; the learner never receives an explicit task identifier.

Evaluation Metric. For a continual-learning run with T tasks we report the *average accuracy* after the final task:

$$\mathsf{Acc}_{\mathsf{avg}} = \frac{1}{T}\sum_{t=1}^{T} \mathsf{Acc}_t,$$

where Acc_t denotes the classification accuracy on the test split of task t evaluated after the model has been trained on all tasks $1, \ldots, T$. Complementary forgetting is measured by

$$\mathcal{F} = \frac{1}{T-1}\sum_{t=1}^{T-1} \big(\max_{k \le t} \mathsf{Acc}_t^{(k)} - \mathsf{Acc}_t^{(T)}\big),$$

with $\mathsf{Acc}_t^{(k)}$ the accuracy on task t after learning task k.

Baselines. All baselines are instantiated with the same backbone (a ResNet-18) and the same optimiser (SGD with momentum 0.9, weight decay 5×10^{-4}). The competitors are:

- **ER** (Experience Replay) [**?**]: stores a small rehearsal buffer (typically 2000 samples) and interleaves replayed samples with current data.
- **DER++** (Data-Efficient Replay) [**?**]: augments ER with a knowledge-distillation term on replayed examples.

Table 1: Average accuracy (%) after the final task on each benchmark. "–" denotes the bufferless variant. The best result per column is bolded.

Method	Split CIFAR-100	5-Dataset	CORe50	Gaussian-Sched. CIFAR-100
ER (buffer)	67.87	71.45	68.21	63.04
DER++ (buffer)	70.12	73.02	70.03	65.18
Co^{2}L (buffer)	71.34	74.10	71.58	66.07
L2P (bufferless)	**83.83**	**79.57**	**76.44**	**71.92**
L2P (with 2000-sample buffer)	85.01	80.73	77.68	73.15

- **Co^{2}L** (Continual Contrastive Learning) [**?**]: combines contrastive regularisation with a replay buffer.

For a fair comparison, we also evaluate L2P in a *bufferless* configuration (Prompt Pool only) and in a *buffer-augmented* configuration (Prompt Pool plus a 2000-sample replay buffer). The former reflects the key claim of L2P: the Prompt Pool can substitute the external memory.

5.2 Results

Table 1 summarises the quantitative outcomes. Several observations are noteworthy:

(a) **Superior bufferless performance.** L2P reaches 83.83% average accuracy on Split CIFAR-100 without any rehearsal buffer, a gain of $+15.96$ points over ER, the strongest buffer-based baseline. Similar margins are observed on the other three benchmarks (ranging from $+5.8$ to $+12.8$ percentage points).

(b) **Robustness to domain shift.** On CORe50 (domain-incremental) and the Gaussian-scheduled task-agnostic CIFAR-100, L2P's prompt retrieval continues to provide relevant contextual cues even when the input distribution drifts, mitigating catastrophic forgetting as reflected by low $\mathcal{F}$ values.

(c) **Marginal benefit from a replay buffer.** Adding a modest rehearsal buffer to L2P yields only a small improvement ($+1.2$ to $+2.2$ points). This confirms that the Prompt Pool already captures a substantial portion of the information traditionally stored in replay buffers.

(d) **Consistency across heterogeneous tasks.** The 5-Dataset sequence mixes grayscale and colour images, simple digits and complex objects. L2P consistently outperforms the baselines, indicating that the query-key mechanism can flexibly adapt its prompt selection to diverse visual semantics.

Questions

1. **Construction of the Split CIFAR-100 benchmark (Easy).**
   Describe how the Split CIFAR-100 benchmark is built for continual-learning evaluation. State the number of tasks, the number of classes per task, and the evaluation protocol used to compute the average accuracy.

2. **Composition and purpose of the 5-Dataset sequence (Easy).**
   List the five datasets that constitute this benchmark (CIFAR-10, MNIST, Fashion-MNIST, SVHN, notMNIST). Explain why combining these heterogeneous datasets is valuable for assessing continual-learning methods, with particular reference to varying data distributions and task similarities.

3. **Domain-incremental learning with CORe50 (Easy).**
   Define a domain-incremental learning scenario. Using CORe50 as an example, describe how the data stream is organised (e.g., different object views, illumination conditions) and why this setting challenges methods that rely on explicit task identity.

4. **Task-agnostic continual learning and Gaussian-scheduled CIFAR-100 (Easy).**
   Explain the notion of a task-free (task-agnostic) continual-learning setting. Detail how the Gaussian-scheduled version of CIFAR-100 is generated and why evaluating L2P under this regime provides insight into its ability to adapt without explicit task boundaries.

5. **Quantitative improvement of L2P over ER (Medium).**
   Using the numbers in Table 1, compute the absolute improvement of **L2P (bufferless)** over **ER (buffer)** on the Split CIFAR-100 benchmark. Discuss why the magnitude of this improvement varies across the four benchmarks (e.g., larger on Split CIFAR-100 than on the 5-Dataset sequence).

6. **Average forgetting computation (Medium).**
   For a continual-learning run with $T = 5$ tasks, the final-task accuracies are $\text{Acc}_1^{(5)} = 78\%$, $\text{Acc}_2^{(5)} = 74\%$, $\text{Acc}_3^{(5)} = 71\%$, $\text{Acc}_4^{(5)} = 68\%$, $\text{Acc}_5^{(5)} = 80\%$. The corresponding maximum accuracies up to each task are 78%, 80%, 81%, 82% respectively.

   (a) Compute the forgetting metric $\mathcal{F}$ using the definition provided in the text.

   (b) Interpret the obtained $\mathcal{F}$ value with respect to the model's stability-plasticity balance.

7. **Prompt Pool as implicit rehearsal (Medium).**
   Explain, referencing the query-key formulation, how the Prompt Pool can serve as an *implicit rehearsal* mechanism. Address the role of: (i) the similarity score s_{ij}, (ii) the aggregation of the top-m values to form the adaptive prompt $\mathbf{p}_i$, and (iii) the prompt-regularisation term in Eq. (2).

8. **Ablation analysis (Medium).**
   Three ablations are reported: (a) removing prompt regularisation ($\lambda = 0$), (b) random retrieval of prompts, and (c) reducing the Prompt Pool size from $K = 200$ to $K = 50$.

   (a) For each ablation, hypothesise the underlying failure mode of L2P (e.g., loss of semantic alignment, increased noise, reduced capacity).

   (b) Propose a diagnostic experiment (including dataset, metric, and expected outcome) that could validate each hypothesis.

9. **Gradient of the loss w.r.t. a prompt value (Hard).**
   The loss $\mathcal{L} = \ell_{\text{CE}}(y_i, \hat{y}_i) + \lambda \|\mathbf{p}_i - \mathbf{v}_{\text{ref}}\|_2^2$ depends on a selected prompt value $\mathbf{v}_j$ through the weighted sum that yields $\mathbf{p}_i$. Derive $\dfrac{\partial \mathcal{L}}{\partial \mathbf{v}_j}$ explicitly (show the contribution of both the task loss and the regularisation term). Discuss how this gradient encourages the Prompt Pool to stay aligned with the evolving query distribution and why such alignment is crucial for buffer-less continual learning.

10. **Designing a task-aware retrieval rule (Hard).**
    Suppose a coarse task identifier is available at test time. Propose a modification to the Prompt Pool retrieval rule that exploits this identifier while preserving the bufferless advantages of L2P. Predict qualitatively how your change would affect (i) average accuracy, (ii) forgetting $\mathcal{F}$, and (iii) the required size of the Prompt Pool.

11. **Lipschitz continuity of the retrieval operator (Hard).**

(a) Briefly define Lipschitz continuity for a neural-network similarity function such as $s_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\|\mathbf{q}_i\|\|\mathbf{k}_j\|}$.

(b) Argue how Lipschitz continuity of the query-key retrieval could contribute to stability in continual learning, especially under distribution shifts like those encountered in CORe50.

12. **Probing what prompts encode in the 5-Dataset sequence (Hard).**
Design a systematic experiment to determine whether the learned prompts capture colour information, domain information, or task-specific semantics. Outline the method you would use to visualise or quantify the prompt embeddings (e.g., t-SNE/UMAP plots, linear probing, correlation with colour histograms) and how the results would support your conclusion.

13. **Privacy and ethical considerations (Hard).**
Discuss the ethical advantages and potential risks of using a Prompt Pool instead of a raw rehearsal buffer in privacy-preserving continual learning. Address at least three points covering data leakage, model-inversion attacks, and compliance with data-protection regulations such as GDPR.